ECSE 526
Assignment #2: Value and Policy Iteration
Eric Xia
260962364

Part 1: To run the code: python a2p1.py

1. **Compute the state utilities and optimal policy using the value iteration and policy iteration algorithms described by Russell and Norvig. Use *r* = -0.04. For value iteration, set the ε parameter value to 0.0001; if the maximum change in utility estimates for any state, between successive iterations, is less than ε(1 - γ)/γ the utilities are considered to have converged.**

The state utilities and optimal policy terminal outputs are contained in Appendix 2.1.

2. Let *π* refer to the policy consisting of going down all the time, wherever the agent is on the board. For this non-optimal policy, compute and show the values of matrix A and vector b for *r* = –0.2 and *r* = –0.4; what do you observe? (important: respect the ordering of the state utilities in *u*).

The values of matrix A and vector b for r = -0.2 and r = -0.4 terminal outputs are contained in Appendix 2.2.

3. Show mathematically that for any policy (optimal or not), the associated state utilities in u vary linearly with *r*. That is, show that u = c + *r*d, where c and d are vectors.

By rearranging the Bellman equation
$U(s) = R(s) + \gamma \Sigma P(s'|s,\pi(s)) U(s')$
For the linear system Au = b:
$U(s) - \gamma \Sigma P(s'|s,\pi(s)) U(s') = R(s)$

When a possible transition leads to a terminal state, as this is a constant, the equation can be further simplified such that the constant can be added to the value of b, where:
$U(s) - \gamma \Sigma P(s'|s,\pi(s)) U(s') = r + \gamma P(s'|s,\pi) R(terminal)$

Where $\gamma P(s'|s,\pi) R(terminal)$ is the result of a possible transition that leads to a terminal state, given by the summation. Thus, we can see that for any policy, vector b is of the form:
$b[i] = R(s) + \gamma P(s'|s,\pi) R(terminal)$, where $\gamma P(s'|s,\pi) R(terminal)$ is a vector of constants.

Additionally, note that when U(s) is a terminal state, U(s) = R(terminal)

Let $t = \gamma P(s'|s,\pi) R(terminal)$, and vector v = 1 if s is a non-terminal state, and v = 0 if s is a terminal state (as r only contributes to b when s is a non-terminal state).
b = r*v + t

As can be seen in [appendix], the matrix A is invertible as A is a square matrix with a determinant > 0. Solving for u by rearranging the equation Au=b:

u = b*A^(-1)
u = A^(-1)(r*v + t)
u = A^(-1)r*v + A^(-1)*t

Let c = A^(-1)*t, and d = A^(-1)*v. As t and v are both vectors, c and d are vectors.
Thus, we can see that u = c + rd.

4. Verify your proof in Question 3 by plotting the value of the utilities of states s_{32}, s_{21} and s_{44} as a function of *r*, where *r* varies between -4 and 0. For this question, use the policy π described in Question 2. Do not forget to label your curves and axes.
Graph is located in Appendix 1.1, Raw data located in ECSE526Graphs.csv.
This utility values were generated using Excel.

5. Using the value iteration algorithm, plot the utilities of states s_{32}, s_{33} and s_{23} as a function of *r*, where *r* varies between -4 and 0. Explain what you see.
Graph is located in Appendix 1.2, Raw data located in ECSE526Graphs.csv. The utility values were generated from the value_iteration_values function.

You can see that the utilities of s32, s33, and s23 have a linear relationship with r when the number of iterations is the same. For example, the values from -4.0<r<-2.7 require 29 iterations before reaching the terminal condition; thus, these contain a linear relationship. However, as r = -2.6 reaches the terminal condition after 28 iterations, the difference increases, thus no longer being linear.

6. Based on what you learned in the previous questions and without computing the optimal policy for any value of *r*, can you determine just by examining your plot in Question 5 whether the optimal policies associated with *r* = –3.5 and *r* = –0.5 are similar or different? Provide adequate details in your response.
Based on the plot, the optimal policies associated with r = -3.5 and r = -0.5 are different. This can be examined by the linearity of the graph. We have seen in Question 3 that when the optimal policy remains constant over all r values, the resulting utility is linear. As stated in Question 5, it can be observed that when the number of iterations of the value iteration algorithm changes, the slope of the curve also changes; thus, as the slope changes at different sections of the curve, this violates the linear utility equation u = c + rd. Therefore, the optimal policy does not stay constant, and would be different with r = -3.5 and r = -0.5.

Part 2: To run the code: a2p2.py
1. How do you represent each state?
There are 9 squares in the one-dimensional world, with 5 possible values for j(t-1). Each state is a vector that contains the square value along with the value of j(t-1) (i.e. square 1 with j(t-1)=1: S(1,1)). Therefore, there are 45 total states in the world.

2. How many non-terminal and terminal states does this problem have? (Give both numbers)

Terminal states: S(3,-2), S(3,-1), S(3,0), S(3,1), S(3,2), S(7,-2), S(7,-1), S(7,0), S(7,1), S(7,2), S(5,0): 11 terminal states
The remaining 34 are non-terminal states.

3. What is the range of possible actions at each state?
The possible actions at each state terminal output is contained in Appendix 2.3.

4. Compute the state utilities and optimal policy using the value iteration and policy iteration algorithms described in Russell and Norvig. Show your results for all states for the following values of *r:* –0.04, –0.5 and –1.
The results for all states for r: -0.04, -0.5, -1 terminal outputs are contained in Appendix 3.1, 3.2, and 3.3 respectively.

Appendix:

AI-Generated Code:

Prompt:

Can you output this data to a csv file?

```
def value_iteration_values(self):
        r_values = np.arange(-4.0, 0.1, 0.1)
        for r in r_values:
            utilities = self.value_iteration(r)
            u_32 = utilities[(3, 3)]
            u_33 = utilities[(2, 3)]
            u_23 = utilities[(2, 2)]
            print(f"r: {r: .1f}, u_32: {u_32}, u_33: {u_33}, u_23:
{u_23}")
```

Output:

```
def value_iteration_values(self):
    import csv
    import datetime

    # Create filename with timestamp
    timestamp = datetime.datetime.now().strftime("%Y%m%d_%H%M%S")
    filename = f"value_iteration_utilities_{timestamp}.csv"

    r_values = np.arange(-4.0, 0.1, 0.1)

    with open(filename, 'w', newline='') as csvfile:
        writer = csv.writer(csvfile)

        # Write header with more descriptive names
        writer.writerow(['r_value', 'U_s33_3_3', 'U_s23_2_3',
'U_s22_2_2'])

        for r in r_values:
            utilities = self.value_iteration(r)
            u_s33 = utilities[(3, 3)]
            u_s23 = utilities[(2, 3)]
            u_s22 = utilities[(2, 2)]

            r_rounded = round(r, 1)
            writer.writerow([r_rounded, u_s33, u_s23, u_s22])
```

```
    print(f"\nValue iteration data saved to: {filename}")
    print("To create plot in Excel:")
    print("1. Open the CSV file")
    print("2. Select all data")
    print("3. Insert → Scatter → Scatter with Smooth Lines")
    print("4. X-axis: r_value, Y-axis: utility values")
```

Prompt:
Print out the results from Value Iteration and Policy Iteration

```
def main():
    """Main function to run both algorithms"""
    grid = Grid()

    # Run Value Iteration - get utilities
    value_iteration = grid.value_iteration(-0.04)

    # Run Policy Iteration - get optimal policy
    policy_iteration = grid.policy_iteration(-0.04)
```

Output:

```
# Result display logic generated by Github Copilot
    print("\nVALUE ITERATION Results:")
    print("=" * 50)
    print("\nState Utilities:")
    print("Row\\Col    1        2        3        4")
    for i in range(1, 5):
        row_str = f"  {i}    "
        for j in range(1, 5):
            if (i, j) in grid.blocked_states:
                row_str += "  BLOCK  "
            else:
                row_str += f"{value_iteration[(i, j)]:8.4f} "
        print(row_str)

    print("\nPOLICY ITERATION Results:")
    print("=" * 50)
    print("\nOptimal Policy:")
    print("Row\\Col  1   2   3   4")
    action_symbols = ['↑', '→', '↓', '←']
```

```
    for i in range(1, 5):
        row_str = f"  {i}   "
        for j in range(1, 5):
            if (i, j) in grid.blocked_states:
                row_str += " ■  "
            elif (i, j) in grid.terminal_states:
                row_str += " T  "
            else:
                action = policy_iteration[(i, j)]
                if action is not None:
                    row_str += f" {action_symbols[action]}  "
                else:
                    row_str += " -  "
        print(row_str)
```

Prompt:
Can you format the matrix display. What method can i use to check if the matrix has a determinant.

```
def display_linear_system_matrices(self):
        print("\nPi Policy")

        for r_val in [-0.2, -0.4]:
            print(f"\n{r_val}")
            A, b = self.pi_policy(r_val)

            print(f"\nMatrix A")
            print(A)
            print(f"\nVector b:")
            print(b)
```

Output:

```
# Display logic generated by GitHub Copilot
    def display_linear_system_matrices(self):
        print("\nPi Policy")

        for r_val in [-0.2, -0.4]:
            print(f"\n{r_val}")
            A, b = self.pi_policy(r_val)
```

```
            # check if A is invertible
            det_A = np.linalg.det(A)
            if abs(det_A) < 1e-10:
                print("Matrix A is singular and cannot be inverted.")
            else:
                print(f"Matrix A is invertible (determinant =
{det_A:.6f}).")

            print(f"\nMatrix A")
            np.set_printoptions(precision=4, suppress=True, linewidth=200)
            print(A)

            print(f"\nVector b:")
            print(b)
```

Prompt:
Can you format the printing of the results?
(*Rest of print_results code*)

```
                print(f" {position} | {prev_jump} | {utility} |
{str(action)} | {str(end_position)} | {state_type} ")
```

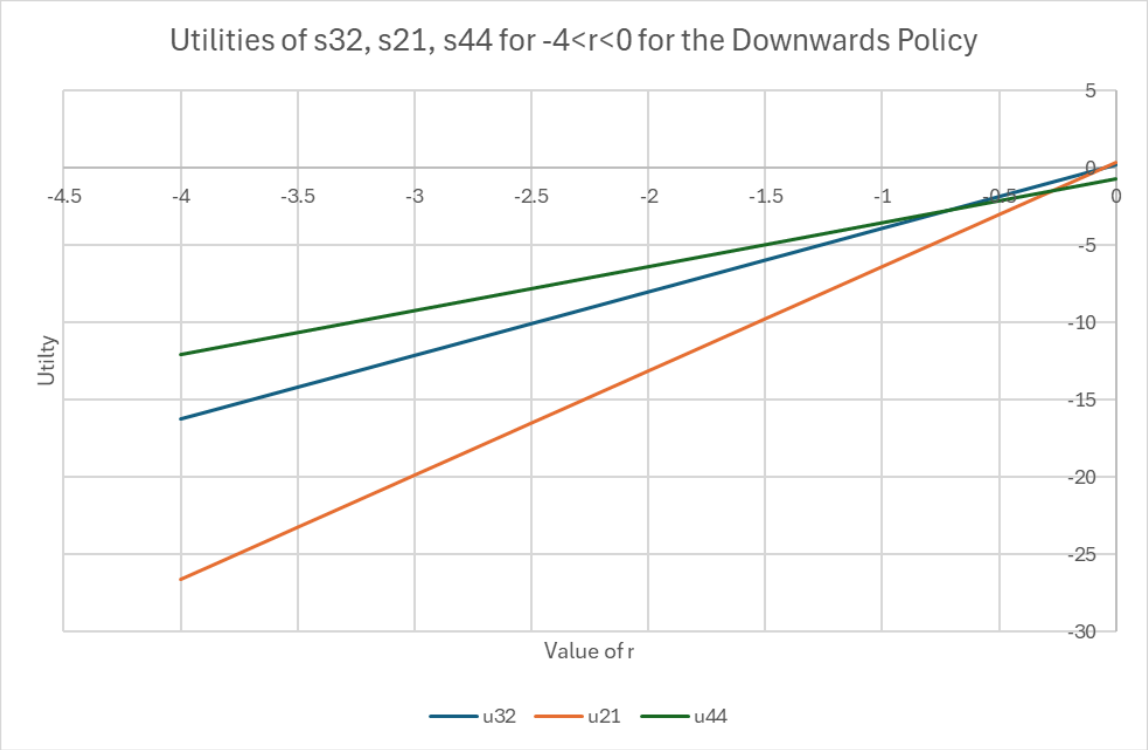
Output:

```
                print(f"    {position}     |      {prev_jump:2d}     |
{utility:8.4f}  | {str(action):>6}         | {str(end_position):>6}       |
{state_type}")
```

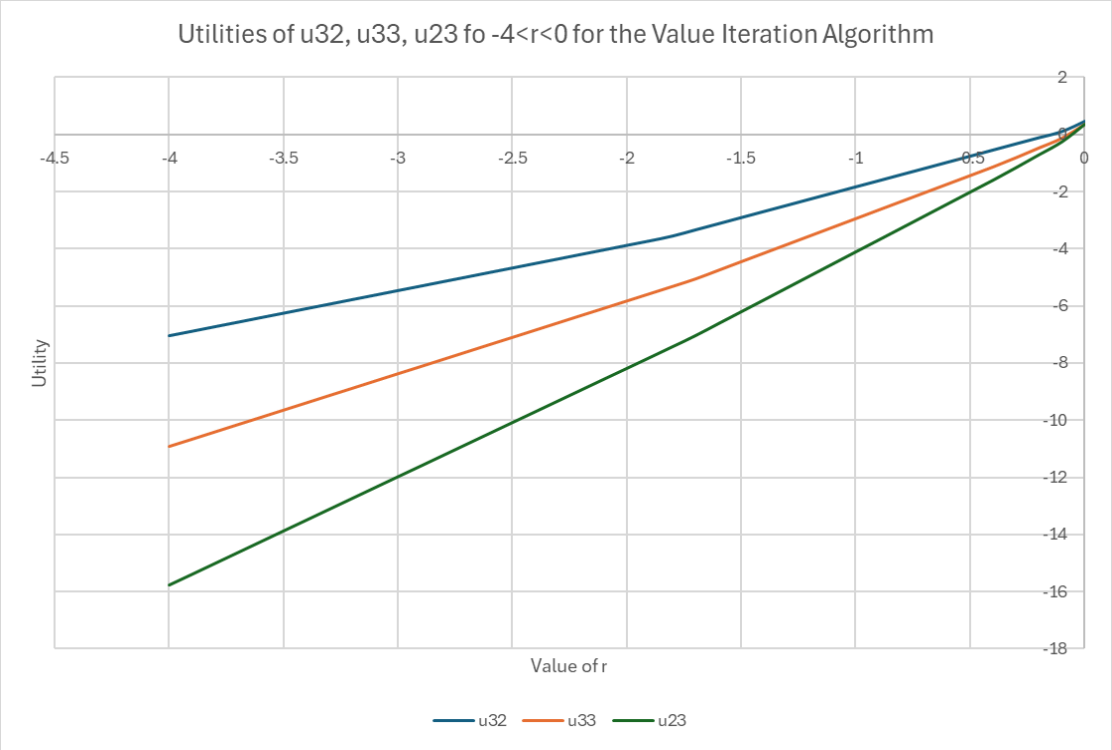
While coding, GitHub Copilot was running in my IDE, Visual Studio Code. There were occasional code suggestions and line completions that were used throughout the program

Graphs:

1.1


Utilities of s32, s21, s44 for -4<r<0 for the Downwards Policy
Utilty
Value of r
u32
u21
u44

1.2


Utilities of u32, u33, u23 fo -4<r<0 for the Value Iteration Algorithm
Utility
Value of r
u32
u33
u23

Terminal Screenshots

2.1

```
VALUE ITERATION Results:
================================================

State Utilities:
Row\Col    1        2        3        4
  1     0.0782   0.0268   BLOCK   -0.1873
  2     0.1688   0.0838   0.1016  -0.1535
  3     0.2834   BLOCK    0.2567  -1.0000
  4     0.3919   0.5486   0.6991   1.0000

POLICY ITERATION Results:
================================================

Optimal Policy:
Row\Col  1   2   3   4
  1      ↓   ←   ■   ↓
  2      ↓   ←   ↓   ↑
  3      ↓   ■   ←   T
  4      →   →   →   T
```

2.2

```
Pi Policy
================================================

r = -0.2
Matrix A is invertible (determinant = 0.002417).

Matrix A
[[ 0.28  0.    0.    0.   -0.18  0.    0.    0.    0.    0.    0.    0.    0.    0.  ]
 [-0.63  0.73  0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.  ]
 [ 0.   -0.63  0.91  0.    0.   -0.18  0.    0.    0.    0.    0.    0.    0.    0.  ]
 [ 0.    0.   -0.63  0.91  0.    0.   -0.18  0.    0.    0.    0.    0.    0.    0.  ]
 [-0.09  0.    0.    0.    0.37  0.    0.   -0.18  0.    0.    0.    0.    0.    0.  ]
 [ 0.    0.   -0.09  0.    0.    0.37  0.    0.    0.   -0.18  0.    0.    0.    0.  ]
 [ 0.    0.    0.   -0.09  0.   -0.63  0.82  0.    0.    0.    0.    0.    0.    0.  ]
 [ 0.    0.    0.    0.   -0.09  0.    0.    0.37  0.    0.    0.    0.    0.    0.  ]
 [ 0.    0.    0.    0.    0.    0.    0.   -0.63  0.91  0.    0.    0.    0.    0.  ]
 [ 0.    0.    0.    0.    0.   -0.09  0.    0.   -0.63  1.    0.    0.   -0.18  0.  ]
 [ 0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    1.    0.    0.    0.  ]
 [ 0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    1.    0.    0.    0.  ]
 [ 0.    0.    0.    0.    0.    0.    0.    0.    0.   -0.09  0.    0.    0.82  0.  ]
 [ 0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.   -0.63  0.73]]

Vector b:
[-0.2  -0.2  -0.2  -0.2  -0.2  -0.2  -0.2  -0.02 -0.38 -0.2   1.   -1.   -0.83 -0.2 ]

r = -0.4
Matrix A is invertible (determinant = 0.002417).

Matrix A
[[ 0.28  0.    0.    0.   -0.18  0.    0.    0.    0.    0.    0.    0.    0.    0.  ]
 [-0.63  0.73  0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.  ]
 [ 0.   -0.63  0.91  0.    0.   -0.18  0.    0.    0.    0.    0.    0.    0.    0.  ]
 [ 0.    0.   -0.63  0.91  0.    0.   -0.18  0.    0.    0.    0.    0.    0.    0.  ]
 [-0.09  0.    0.    0.    0.37  0.    0.   -0.18  0.    0.    0.    0.    0.    0.  ]
 [ 0.    0.   -0.09  0.    0.    0.37  0.    0.    0.   -0.18  0.    0.    0.    0.  ]
 [ 0.    0.    0.   -0.09  0.   -0.63  0.82  0.    0.    0.    0.    0.    0.    0.  ]
 [ 0.    0.    0.    0.   -0.09  0.    0.    0.37  0.    0.    0.    0.    0.    0.  ]
 [ 0.    0.    0.    0.    0.    0.    0.   -0.63  0.91  0.    0.    0.    0.    0.  ]
 [ 0.    0.    0.    0.    0.   -0.09  0.    0.   -0.63  1.    0.    0.   -0.18  0.  ]
 [ 0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    1.    0.    0.    0.  ]
 [ 0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    1.    0.    0.    0.  ]
 [ 0.    0.    0.    0.    0.    0.    0.    0.    0.   -0.09  0.    0.    0.82  0.  ]
 [ 0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.   -0.63  0.73]]

Vector b:
[-0.4  -0.4  -0.4  -0.4  -0.4  -0.4  -0.4  -0.22 -0.58 -0.4   1.   -1.   -1.03 -0.4 ]
```

3.1

```
Available Actions for Each State:
 1 | -2 | [-2, -1]
 1 | -1 | [-2, -1, 0]
 1 |  0 | [-1, 0, 1]
 1 |  1 | [0, 1, 2]
 1 |  2 | [1, 2]
 2 | -2 | [-2, -1]
 2 | -1 | [-2, -1, 0]
 2 |  0 | [-1, 0, 1]
 2 |  1 | [0, 1, 2]
 2 |  2 | [1, 2]
 3 | -2 | []
 3 | -1 | []
 3 |  0 | []
 3 |  1 | []
 3 |  2 | []
 4 | -2 | [-2, -1]
 4 | -1 | [-2, -1, 0]
 4 |  0 | [-1, 0, 1]
 4 |  1 | [0, 1, 2]
 4 |  2 | [1, 2]
 5 | -2 | [-2, -1]
 5 | -1 | [-2, -1, 0]
 5 |  0 | []
 5 |  1 | [0, 1, 2]
 5 |  2 | [1, 2]
 6 | -2 | [-2, -1]
 6 | -1 | [-2, -1, 0]
 6 |  0 | [-1, 0, 1]
 6 |  1 | [0, 1, 2]
 6 |  2 | [1, 2]
 7 | -2 | []
 7 | -1 | []
 7 |  0 | []
 7 |  1 | []
 7 |  2 | []
 8 | -2 | [-2, -1]
 8 | -1 | [-2, -1, 0]
 8 |  0 | [-1, 0, 1]
 8 |  1 | [0, 1, 2]
 8 |  2 | [1, 2]
 9 | -2 | [-2, -1]
 9 | -1 | [-2, -1, 0]
 9 |  0 | [-1, 0, 1]
 9 |  1 | [0, 1, 2]
 9 |  2 | [1, 2]
```

3.2

Results for r = -0.04:

State Utilities and Optimal Policy:

Position	Previous Jump	Utility	Optimal Policy	End Position	State Type
1	-2	0.3440	-1	1	Normal
1	-1	0.4267	0	1	Normal
1	0	0.5185	1	2	Normal
1	1	0.5185	1	2	Normal
1	2	0.5185	1	2	Normal
2	-2	0.3440	-1	1	Normal
2	-1	0.3440	-1	1	Normal
2	0	0.3440	-1	1	Normal
2	1	0.6206	2	4	Normal
2	2	0.6206	2	4	Normal
3	-2	-1.0000	Terminal	N/A	Hole
3	-1	-1.0000	Terminal	N/A	Hole
3	0	-1.0000	Terminal	N/A	Hole
3	1	-1.0000	Terminal	N/A	Hole
3	2	-1.0000	Terminal	N/A	Hole
4	-2	0.2696	-2	2	Normal
4	-1	0.6206	0	4	Normal
4	0	0.7340	1	5	Normal
4	1	0.7340	1	5	Normal
4	2	0.7340	1	5	Normal
5	-2	0.5185	-1	4	Food (not eaten)
5	-1	0.8600	0	5	Food (not eaten)
5	0	1.0000	Terminal	N/A	Food
5	1	0.8600	0	5	Food (not eaten)
5	2	0.5185	1	6	Food (not eaten)
6	-2	0.7340	-1	5	Normal
6	-1	0.7340	-1	5	Normal
6	0	0.7340	-1	5	Normal
6	1	0.6206	0	6	Normal
6	2	0.2696	2	8	Normal
7	-2	-1.0000	Terminal	N/A	Hole
7	-1	-1.0000	Terminal	N/A	Hole
7	0	-1.0000	Terminal	N/A	Hole
7	1	-1.0000	Terminal	N/A	Hole
7	2	-1.0000	Terminal	N/A	Hole
8	-2	0.6206	-2	6	Normal
8	-1	0.6206	-2	6	Normal
8	0	0.3440	1	9	Normal
8	1	0.3440	1	9	Normal
8	2	0.3440	1	9	Normal
9	-2	0.5185	-1	8	Normal
9	-1	0.5185	-1	8	Normal
9	0	0.5185	-1	8	Normal
9	1	0.4267	0	9	Normal
9	2	0.3440	1	9	Normal

3.3

Results for r = -0.5:

State Utilities and Optimal Policy:

Position	Previous Jump	Utility	Optimal Policy	End Position	State Type
1	-2	-1.8114	-1	1	Normal
1	-1	-1.4571	0	1	Normal
1	0	-1.0634	1	2	Normal
1	1	-1.0634	1	2	Normal
1	2	-1.0634	1	2	Normal
2	-2	-1.8114	-1	1	Normal
2	-1	-1.7600	0	2	Normal
2	0	-1.4000	1	3	Normal
2	1	-0.6260	2	4	Normal
2	2	-0.6260	2	4	Normal
3	-2	-1.0000	Terminal	N/A	Hole
3	-1	-1.0000	Terminal	N/A	Hole
3	0	-1.0000	Terminal	N/A	Hole
3	1	-1.0000	Terminal	N/A	Hole
3	2	-1.0000	Terminal	N/A	Hole
4	-2	-1.4000	-1	3	Normal
4	-1	-0.6260	0	4	Normal
4	0	-0.1400	1	5	Normal
4	1	-0.1400	1	5	Normal
4	2	-0.1400	1	5	Normal
5	-2	-1.0634	-1	4	Food (not eaten)
5	-1	0.4000	0	5	Food (not eaten)
5	0	1.0000	Terminal	N/A	Food
5	1	0.4000	0	5	Food (not eaten)
5	2	-1.0634	1	6	Food (not eaten)
6	-2	-0.1400	-1	5	Normal
6	-1	-0.1400	-1	5	Normal
6	0	-0.1400	-1	5	Normal
6	1	-0.6260	0	6	Normal
6	2	-1.4000	1	7	Normal
7	-2	-1.0000	Terminal	N/A	Hole
7	-1	-1.0000	Terminal	N/A	Hole
7	0	-1.0000	Terminal	N/A	Hole
7	1	-1.0000	Terminal	N/A	Hole
7	2	-1.0000	Terminal	N/A	Hole
8	-2	-0.6260	-2	6	Normal
8	-1	-0.6260	-2	6	Normal
8	0	-1.4000	-1	7	Normal
8	1	-1.7600	0	8	Normal
8	2	-1.8114	1	9	Normal
9	-2	-1.0634	-1	8	Normal
9	-1	-1.0634	-1	8	Normal
9	0	-1.0634	-1	8	Normal
9	1	-1.4571	0	9	Normal
9	2	-1.8114	1	9	Normal

3.4

Results for r = -1.0:

State Utilities and Optimal Policy:

Position	Previous Jump	Utility	Optimal Policy	End Position	State Type
1	-2	-4.0951	-1	1	Normal
1	-1	-3.4390	0	1	Normal
1	0	-2.7100	1	2	Normal
1	1	-1.9000	2	3	Normal
1	2	-1.9000	2	3	Normal
2	-2	-4.0951	-1	1	Normal
2	-1	-2.7100	0	2	Normal
2	0	-1.9000	1	3	Normal
2	1	-1.9000	1	3	Normal
2	2	-1.9000	1	3	Normal
3	-2	-1.0000	Terminal	N/A	Hole
3	-1	-1.0000	Terminal	N/A	Hole
3	0	-1.0000	Terminal	N/A	Hole
3	1	-1.0000	Terminal	N/A	Hole
3	2	-1.0000	Terminal	N/A	Hole
4	-2	-1.9000	-1	3	Normal
4	-1	-1.9000	-1	3	Normal
4	0	-1.0900	1	5	Normal
4	1	-1.0900	1	5	Normal
4	2	-1.0900	1	5	Normal
5	-2	-1.9000	-2	3	Food (not eaten)
5	-1	-0.1000	0	5	Food (not eaten)
5	0	1.0000	Terminal	N/A	Food
5	1	-0.1000	0	5	Food (not eaten)
5	2	-1.9000	2	7	Food (not eaten)
6	-2	-1.0900	-1	5	Normal
6	-1	-1.0900	-1	5	Normal
6	0	-1.0900	-1	5	Normal
6	1	-1.9000	1	7	Normal
6	2	-1.9000	1	7	Normal
7	-2	-1.0000	Terminal	N/A	Hole
7	-1	-1.0000	Terminal	N/A	Hole
7	0	-1.0000	Terminal	N/A	Hole
7	1	-1.0000	Terminal	N/A	Hole
7	2	-1.0000	Terminal	N/A	Hole
8	-2	-1.9000	-1	7	Normal
8	-1	-1.9000	-1	7	Normal
8	0	-1.9000	-1	7	Normal
8	1	-2.7100	0	8	Normal
8	2	-4.0951	1	9	Normal
9	-2	-1.9000	-2	7	Normal
9	-1	-1.9000	-2	7	Normal
9	0	-2.7100	-1	8	Normal
9	1	-3.4390	0	9	Normal
9	2	-4.0951	1	9	Normal